

Technische Beschreibung

Aufbau

Teil	Technik
Server	Python 3.12, FastAPI, Uvicorn; läuft als Hintergrundprozess auf einem Windows-PC
Datenbank	SQLite (eine Datei <code>daten/lager.db</code> , WAL-Modus), Zugriff über SQLAlchemy Core
Oberfläche	Server-generierte Seiten (Jinja2), HTMX, Alpine.js, Tailwind CSS; alle Dateien lokal, kein Internet nötig
Scanner-App	<code>/m</code> , installierbare Web-App (Manifest, Service Worker <code>/m/sw.js</code>); Scantaste über Tastatureingabe (DataWedge), Kamera-Scan über BarcodeDetector bzw. ZXing (lokal)
Offline-Modus	Warteschlange in IndexedDB des Geräts, Artikelkatalog <code>/m/katalog.json</code> zwischengespeichert, Übertragung über <code>/m/sync</code> mit UUID je Buchung (Tabelle <code>sync_log</code> verhindert Doppelbuchungen)
HTTP / HTTPS	HTTP (Port 8080) lauscht nur auf <code>127.0.0.1</code> – für den Browser am Lager-PC. Alle anderen Geräte nur über HTTPS (Port 8443). Abschaltbar mit <code>http_nur_lokal = false</code> (nicht empfohlen).
Zertifikate	Eigene kleine CA (<code>daten/lager_ca.crt</code> , Download <code>/zertifikat.crt</code> ; der private Schlüssel <code>daten/lager_ca_schluesel.pem</code> wird nie ausgeliefert). Die CA ist per <i>Name Constraints</i> (kritisch) auf <code>10.0.0.0/8</code> , <code>172.16.0.0/12</code> , <code>192.168.0.0/16</code> , <code>127.0.0.0/8</code> , <code>169.254.0.0/16</code> und die Namen dieses PCs beschränkt – auch mit gestohlenem Schlüssel lassen sich keine Zertifikate für fremde Domains ausstellen. Das Serverzertifikat wird beim Start erneuert, wenn sich IP/PC-Name geändert haben oder es in weniger als 30 Tagen abläuft; öffentliche IP-Adressen werden nicht aufgenommen.
Etikettendruck	TSPL direkt an HPRT HT100 (Windows-Spooler RAW über <code>winspool.drv</code> oder TCP 9100); Fallback Browserdruck
Sicherung	täglich ZIP mit konsistenter SQLite-Kopie (Backup-API) und Anhängen

Gleichzeitige Buchungen (PC, mehrere Handscanner, API) werden über Schreibtransaktionen mit sofortiger Sperre (`BEGIN IMMEDIATE`) nacheinander ausgeführt; Lesen blockiert nicht. Getestet mit parallelen Entnahmen des letzten Teils (Threads und echte HTTP-Anfragen).

Anmeldung: Passwörter mit mindestens 10 Zeichen, gespeichert als PBKDF2-SHA256 (240.000 Runden). Nach 5 Fehlversuchen je Konto und Gerät bzw. 20 je Gerät ist die Anmeldung 15 Minuten gesperrt (nur im Speicher; Neustart hebt die Sperre auf). Der erste Administrator kann nur am Lager-PC selbst angelegt werden. Die Sitzung liegt signiert im Cookie (30 Tage; über HTTPS mit `Secure`, immer `HttpOnly`, `SameSite=Lax`), wird aber bei jedem Aufruf gegen die Tabelle `users` geprüft – Sperre, Rollenwechsel und Passwortänderung wirken sofort. Größere Daten (Sammelentnahme-Liste, Übernahmebericht) liegen in der Tabelle `settings`, nicht im Cookie (Browsergrenze 4 KB). Weiterleitungen (`weiter`, Referer) sind auf Pfade der eigenen Anwendung beschränkt; Anhänge werden nur als Bild/PDF/Text im Browser angezeigt, alles andere als Download.

Datenmodell

`artikel`, `lagerplaetze`, `bestand` (Menge je Artikel und Platz), `bewegungen` (jede Buchung mit Zeit, Benutzer, Menge $\pm$, Bestand danach, Empfänger, Kostenstelle, Zweck, Quelle `pc/mobil/offline/api/import`), `reservierungen`, `bestellungen`, `inventuren/inventur_pos`, `lieferanten`,

anhaenge , users , audit , settings , sync_log .

Mengen: Komma oder Punkt als Dezimaltrenner; „nan“, „inf“ und Werte über 1 Milliarde werden abgelehnt.

Buchungsarten: eingang , ausgang , umbuchung (zwei verknüpfte Zeilen), inventur , anlage , ausleihe , rueckgabe , info (nur Historie aus dem Altsystem).

Alle Bestandsänderungen laufen über app/services/lager.py (Klasse Lager).

Übernahme aus Casper

app/services/casper_import.py liest den HeidiSQL-Export (nur INSERT-Anweisungen am Zeilenanfang – das INSERT im Casper-Trigger bewegungsdaten_after_insert wird ignoriert) und übernimmt:

Casper	Neu
stammdaten.Artikelnummer / Bezeichnung	artikel.nummer / bezeichnung
Meldebestand / Mindestbestand (Text, 0 = nicht gepflegt)	Zahl oder leer
Freifeld1 „Lieferant“	lieferanten (Schreibvarianten zusammengeführt)
Freifeld2 „Gruppe“, Freifeld3 „Typenbezeichnung“	kategorie, typ
Freifeld4 „Austragungsgrund“, Freifeld5	notiz
lagerorte (Leerzeichen bereinigt, Duplikate summiert, leer → OHNE-PLATZ)	lagerplaetze + bestand
bewegungsdaten (Eingang, Ausgang, Umbuchung/Umlagerung, Inventur, Neuer Artikel, Artikel bearbeitet/gelöscht)	bewegungen (quelle = import)
benutzer	users (gesperrt, ohne Passwort)

Lagerort-Zeilen zu nicht mehr vorhandenen Artikeln werden nicht übernommen (im Bericht genannt).

Schnittstelle

Token-Authentifizierung (Authorization: Bearer <Schlüssel> , Schlüssel je Benutzer unter *Einstellungen* → *Schnittstelle*). Endpunkte unter /api/v1/... : Artikel (Liste, Detail mit Beständen je Platz), Lagerplätze, Bewegungen seit ID, Buchung anlegen, Reservierung anlegen. Beschreibung maschinenlesbar unter /api/openapi.json . Gedacht für die spätere Anbindung der Instandhaltungs-Software (Ersatzteile reservieren/entnehmen je Auftrag).

Entwicklung

```
pip install -r requirements.txt
cd frontend && npm install && ./build_css.sh
python -m app.main # http://localhost:8080 (nur lokal), https://<PC>:8443
python -m pytest tests # LV_CASPER_EXPORT=<datei.sql> testet zusätzlich die
Übernahme echter Daten (Sollwerte 821 / 377 / 2702)
```